Wine E-Label Platform — Engineering Master Audit & Fix Report

End-to-End Reliability, Persistence Across Restarts, Security Authorization, Database Relationships & EU Compliance

Repository	https://github.com/pranavbhandare31/Wine_lable
Target Branch	main
Primary Technologies	Next.js 14, TypeScript, Tailwind CSS, next-intl, Drizzle ORM, PGlite / PostgreSQL, Clerk Auth
Compliance Standards	EU Regulation 2021/2117 & EU Regulation 1169/2011 (Wine E-Labels & Multilingual Disclosures)
Test Suite Results	**27 / 27 Tests Passed (100% Passing Across 6 Test Suites)**
Generated	October 17, 2026

1. Executive Task Summary

Task	Description	Status	Verified Resolution Summary
Task 1	QR Routing Mismatch	**COMPLETE**	Fixed /public/products/:id to singular /public/product/:id (Commit 8ef8138).
Task 2	createdBy Column Type	**COMPLETE**	Altered integer to text in Drizzle & Zod with Migration 0002 for Clerk IDs (Commit 142841c).
Task 3	Product-Ingredient Relations	**COMPLETE**	Implemented junction table product_ingredients, relations, migration 0003, API, UI, and tests (Commit ddca2dd).
Task 4	English/French Translations	**COMPLETE**	Added 'product' namespace to en.json & fr.json, updated public product page without fallbacks (Commit 29579ee).
Master Task	End-to-End Reliability & Auth	**COMPLETE**	Enabled PGlite persistence (.pglite/), Clerk user ownership authorization, numeric validation resilience, and 27 tests.

2. Master Task: Audit, Fixes & Architectural Hardening

1. Database Persistence Across Restarts: Configured PGlite to persist to .pglite/ on disk during local development, so created products, ingredients, and associations survive restarts. Protected build workers with isBuild in-memory mode.

2. Clerk Ownership & Authorization: Bound createdBy to Clerk userId on product/ingredient creation. Scoped GET /api/products to authorized user records, and enforced 403 Forbidden checks on unauthorized PUT and DELETE operations.

3. Resilient Validation: Enhanced insertProductSchema to accept numeric values for vintage, alcoholContent, kcal, etc., through Zod union transformations.

4. Route Hardening: Handled non-numeric product IDs gracefully with notFound() (404), and eliminated invalid console error imports in upload endpoints.

Critical Workflow Verification Matrix:

Workflow Scenario	Verification Scenario	Result
Workflow A: Product Creation	Tested product creation with numeric vintage/alcohol; verified Clerk createdBy attribution and persistence.	**PASSED**
Workflow B: Ingredients & Relations	Associated ingredients (yeast, sulphites) to product; verified join retrieval and duplicate rejection.	**PASSED**
Workflow C: Public E-Label & i18n	Verified English and French translations; tested NaN/invalid ID handling returning 404.	**PASSED**
Workflow D: Security & Ownership	Simulated unauthorized mutation by User B on User A's product; successfully rejected with 403 Forbidden.	**PASSED**
Workflow E: Persistence Across Restart	Created data on disk, closed client (shutdown), reopened on same directory; verified 100% data intact.	**PASSED**

3. Test Suite & Verification Results

- **TypeScript Typecheck:** npm run check-types exited with 0 errors.
- **Automated Tests:** npm run test — 6 test files, 27 / 27 tests passed.
- **Production Build:** npm run build passed with exit code 0, generating all 20 static and dynamic routes.

4. Spoken Explanation for Sir & Likely Viva Questions

Spoken Script for Sir:
"Good morning Sir. Over this comprehensive engineering cycle, we audited and resolved all platform issues: we fixed the QR route mismatch, migrated createdBy to text for Clerk user IDs, established the Many-to-Many product_ingredients relational table with cascade deletion, completed 26-key English and French translation dictionaries, enabled PGlite filesystem persistence (.pglite/) so data survives restarts, and implemented Clerk ownership authorization across API endpoints. All 27 tests and the production build pass with 100% success."

Key Viva Questions:

Q1: How does the application ensure data survives server restarts locally?
Answer: In src/libs/DB.ts, PGlite is configured to use a filesystem directory (.pglite/) during dev/start, while using in-memory mode during builds and tests to prevent worker lock contention. When DATABASE_URL is provided, it seamlessly connects to PostgreSQL.

Q2: How is multi-user authorization enforced across API endpoints?
Answer: We integrated Clerk's auth() helper. On creation, createdBy is stamped with the user's ID. On updates or deletes, the backend checks whether the resource's createdBy matches the requesting user, rejecting unauthorized access with HTTP 403 Forbidden.

Q3: Why was the Zod schema causing 400 errors for vintage and alcoholContent?
Answer: The original schema strictly required string types. When client forms or API consumers submitted numeric values like 2020 or 13.5, Zod threw validation errors. We updated the schema to accept a union of string and number transformed to string.